flashjet

GPU-native jet clustering with the full merge history

Chirayu Gupta

ML4Jets 2026 · Vienna

September 2026

Benchmarks: **ATLAS Open Data** (CC0). Closures: analytic predictions on toy showers.

Outline

Jet clustering, and flashjet

What jet clustering is, and why it is everywhere

A quark or gluon produced in a collision is never seen directly: it radiates and hadronises into a **spray of order 10–100 particles**. A *jet algorithm* groups those particles back together so the spray can be treated as one object with a momentum and a mass.

The LHC standard is the **generalised-k_t** family — anti-k_t, k_t, Cambridge–Aachen — implemented in **FastJet**. It is **sequential recombination**: repeatedly merge the closest pair under a distance measure, until nothing is left.

Essentially every LHC measurement that involves hadrons runs this step, often several times per event, and again for every systematic variation.

Where the cost shows up

- **reconstruction** — every event, every reprocessing
- **analysis** — reclustering under many variations
- **substructure** — a second pass to groom or find subjets
- **ML pipelines** — jets rebuilt on every pass over the data

The algorithms are inherently sequential and have always run on the **CPU**.

The opportunity: the data is already on the GPU

Clustering is a **CPU step**, and increasingly it is the *only* CPU step in a workflow that otherwise lives on an accelerator. Four-momenta are copied to the host, clustered, and copied back.

That round trip is paid over and over when

- jets are **reclustered** under many systematic variations,
- the same events are revisited **many times** — as in training a network,
- clustering must sit **inside** a differentiable pipeline.

It is also simply a large fraction of reconstruction time, entirely independent of any machine learning.

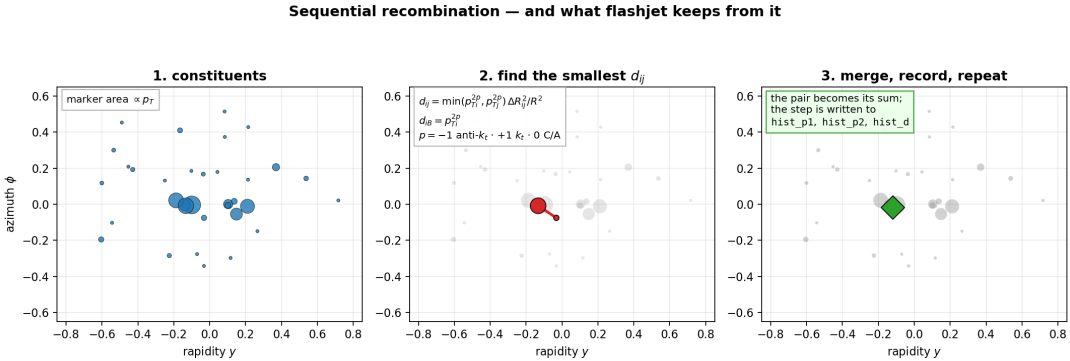
flashjet

Implements anti-k_t, k_t and Cambridge–Aachen **natively on the GPU**.

A batch of jets or events is clustered in device memory, and the data never leaves it.

Triton/PyTorch. One entry point; runs unchanged on CPU and GPU.

How sequential recombination works

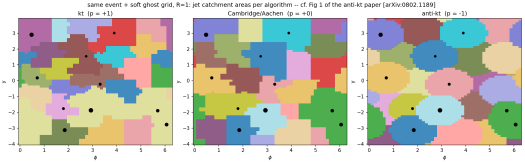


Repeat until nothing is left: if the smallest distance is a d_{ij}, merge that pair; if it is a d_{iB}, call that a jet and remove it.

One exponent p selects the algorithm — the *same* kernel serves all three.

One exponent, three algorithms

p	algorithm	merges first	gives
-1	anti-k_t	around the **hardest** particles	rigid, circular jets
0	Cambridge–Aachen	the **closest** pair	angular-ordered tree
$+1$	k_t	the **softest** pair	k_t-ordered tree



The ordering is not a detail — it is what the substructure reads.

- anti-k_t gives the *jets*: catchment areas $\to \pi R^2$ — the defining rigid-cone result, from flashjet's own clustering.
- **C/A** orders by angle $\Rightarrow$ the natural tree for **grooming** and the **Lund plane**.
- k_t orders by relative transverse momentum $\Rightarrow$ the natural tree for **exclusive subjets**.

So flashjet keeps whichever history the observable needs — and all three cost the same.

It returns the tree, not just the jets

Clusters a **batch of jets or events at once**, and returns **two** things:

1. the particle → jet assignment;
2. the **complete merge history** — which pair merged, into what, at what distance.

Why the history matters

Groomed mass, z_g, R_g, Lund coordinates, exclusive subjets become **reads of that tree**.

No second clustering pass.

`jet_idx`, `n_jets`	particle-to-jet assignment
`hist_p1`, `hist_p2`, `hist_child`	which pair merged into what
`hist_d`	the d_{ij} at each merge step

One entry point

```
out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
#   p4        (B, N, 4) torch tensor   |  mask  (B, N) bool
#   algorithm "antikt" | "kt" | "cambridge"

out.n_jets                                          # jets found
out.jets_p4(p4)                                     # jet four-momenta
out.exclusive_jets(n_jets=3)                        # F1
out.groomed_jets(p4, R, z_cut=0.1, beta=0.0)        # F2  soft drop / mMDT
out.lund_coordinates(p4, R)                         # F3  primary Lund plane
```

Same call on CPU and GPU. `jets_p4` is a `scatter_add` of constituents, so **gradients flow through the clustering**.

What is implemented

Clustering

- generalised-k_t: anti-k_t, k_t, C/A
- batched, ragged input via a mask
- three backends, auto-dispatched by size
- merge history recorded in-kernel
- jet regime *and* event regime

Substructure — reads of the history

- **F1** exclusive k_t subjets
- **F2** soft drop / mMDT / mass-drop
- **F3** primary Lund coordinates

All three are pure-torch post-reads: no kernel changes, CPU/GPU identical, negligible cost.

Status

A **prototype** — the physics is validated and the speedups are real, but it is a Python/Triton library, **not** integrated into any experiment's reconstruction framework.

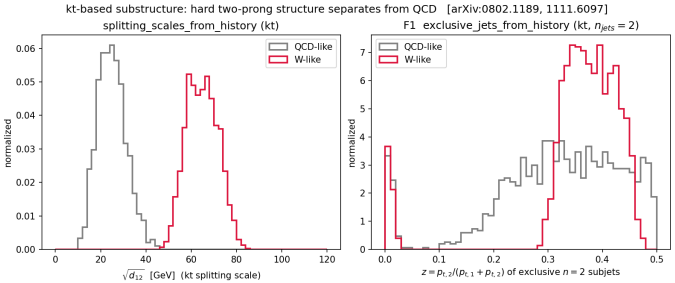
F1 — exclusive subjets from the k_t history

What it does. Undo the last merges of the sequence to expose exactly n subjets — or stop at a d_{cut}.

How. The history is already a binary tree. Walk back up it $n-1$ steps; no reclustering.

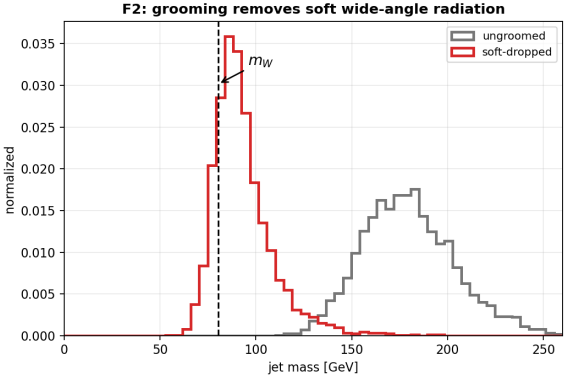
Why k_t. k_t merges the softest pair first, so the *last* merges are the hard prongs — exactly what a 2-prong decay leaves behind.

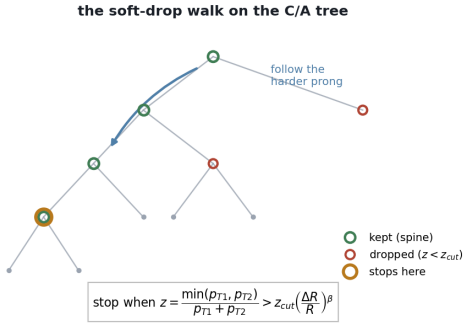
The splitting scales $\sqrt{d_{12}}, \sqrt{d_{23}}, \ldots$ fall out of the same array.



Toy W-like (2-prong) vs QCD-like fat jets. **Left:** the k_t splitting scale $\sqrt{d_{12}}$ separates cleanly. **Right:** the momentum balance z of the two exclusive subjets — the W sits near $z \sim 0.4$, QCD spreads to small z.

F2 — soft drop / mMDT grooming from the C/A history

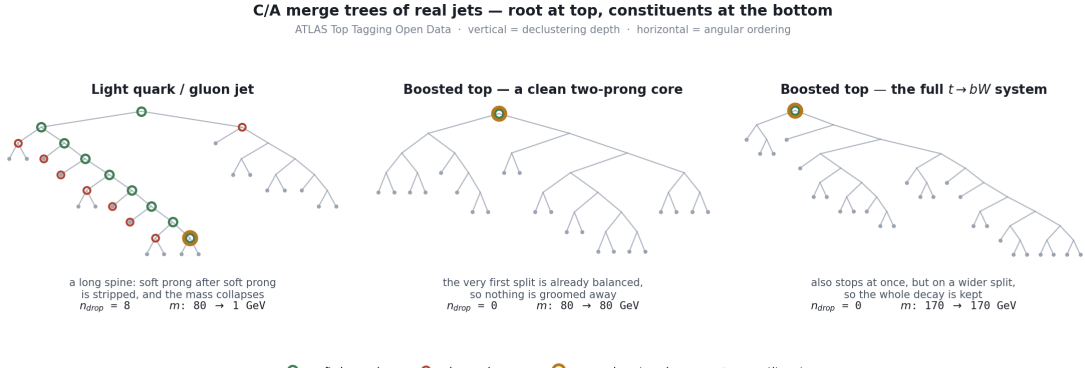




Walk down the C/A tree along the harder branch, discarding the softer prong until the momentum-sharing condition is met. $\beta = 0$ recovers mMDT.

The walk is $O(\log_2 n)$ on the stored tree — it never touches the constituents again, so grooming is essentially free once clustered.

What the tree looks like on real jets



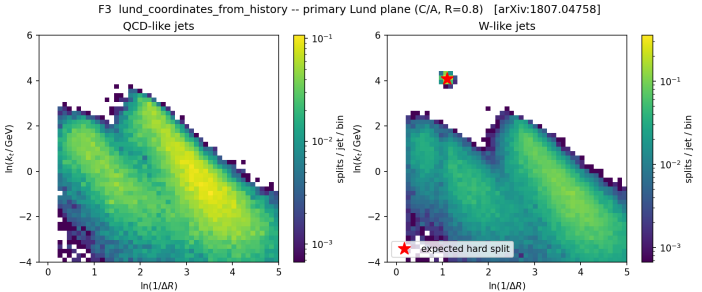
Three jets from ATLAS Open Data, clustered with C/A and groomed with soft drop ($z_{cut} = 0.1$, $\beta = 0$). **Left:** a light-quark/gluon jet is a long *staircase* — there is no balanced split to stop on, so prong after prong is stripped and the mass collapses to nothing. **Centre and right:** boosted top jets stop *immediately*, because the very first split is already hard and balanced. **The number of drops is itself a discriminant** — and it is read straight off the stored tree, at no extra cost.

F3 — primary Lund coordinates from the C/A history

What it does. For every primary splitting, emit $(z,\ \Delta R,\ k_t,\ \ln 1/\Delta R,\ \ln k_t,\ d)$.

How. One pass down the primary branch of the C/A tree; each node is one point in the plane.

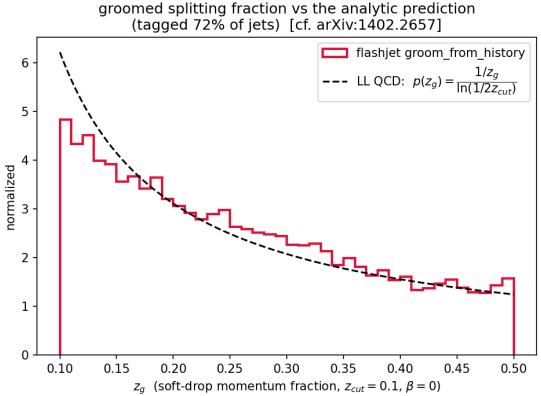
Consistency. The d-channel ties *exactly* to the splitting scales computed independently — the same number reached two ways.



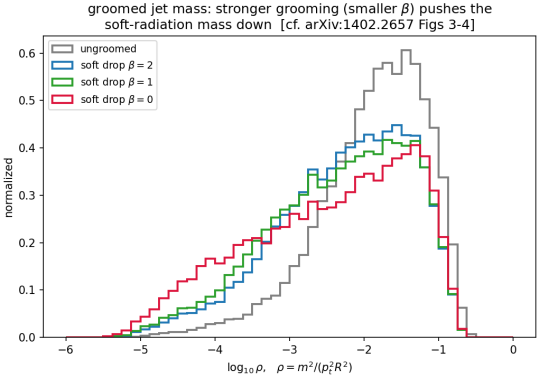
QCD fills the soft-collinear region smoothly; the W-like sample adds a **localised hard splitting** exactly where the two-prong decay is predicted to sit (star).

Correctness

The history is right: closures against analytic predictions



Soft-drop z_g follows the $1/z$ form predicted at leading-log, recovered from the C/A history.

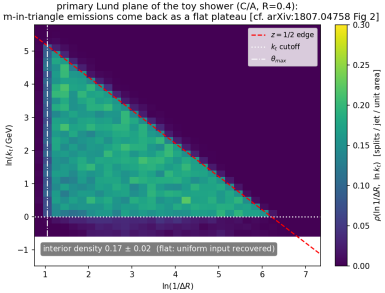


Groomed-mass ordering in β: larger β grooms less, so the mass peak moves up.

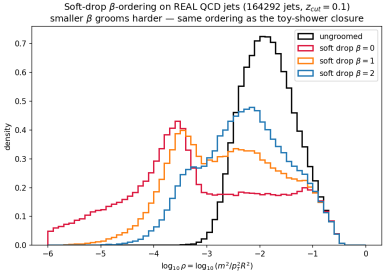
Toy-shower input, so the *prediction* is unambiguous — these test the decoders, not an event generator.

Clustering geometry

anti-k_t jet areas $\to \pi R^2$.



Lund plane uniform inside the kinematic triangle.



β-family ordering.

Independent NumPy tree-walks pin every decoder; the d-channel of the Lund output ties exactly to the splitting scales.

Same jets as FastJet — across the whole grid

3 algorithms $\times$ **5 radii** $\times$ **5 multiplicity bins** $\times$ **2 samples**, 20 000 jets per bin, on ATLAS Open Data.

check	**result**
number of jets found	**100.000 %** at every one of 150 points
leading-jet p_T within 10^{-4}	1.0000 at 148/150 points
median relative difference	$\sim 7 \times 10^{-8}$

k_t and C/A had never been checked against FastJet on real detector data — only against NumPy tree-walks. They now agree across the full radius $\times$ multiplicity grid.

The two exceptions are the *same single jet*: one constituent assigned across an R boundary. Jet counts still match and the leading jet is never ambiguous.

Same jets as the experiment

A stronger test than agreeing with FastJet.

ATLAS publishes an open dataset carrying both the **calorimeter clusters** and **its own reconstructed jets**. Cluster the ∼600 clusters per event ourselves, and compare:

n_{jets} vs ATLAS	**100.0 %** match
mean jets/event	10.96 vs 10.96

This closes the algorithm against **the experiment's own published output** — not against another implementation of the same algorithm.

599.3 clusters/event: the event regime, where the problem is $O(N^2)$.

Speed

The benchmark

Data — ATLAS Open Data, freely presentable

sample	jets	$\langle n_{\text{const}} \rangle$
boosted top	30 000	59.2
QCD (light q/g)	30 000	52.4

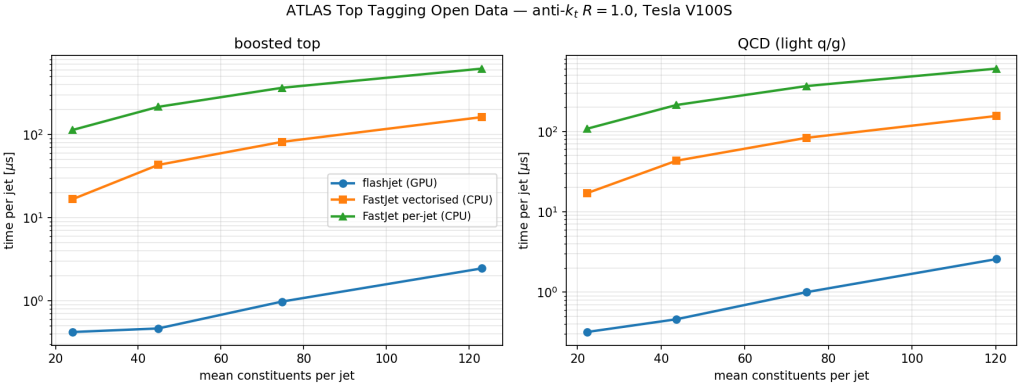
Large-R jets with calorimeter-cluster constituents.

Method

1. extract constituents once, save;
2. cluster on GPU, time it, save the *same* events;
3. cluster those saved events with FastJet;
4. **check they agree** — else the timing is meaningless.

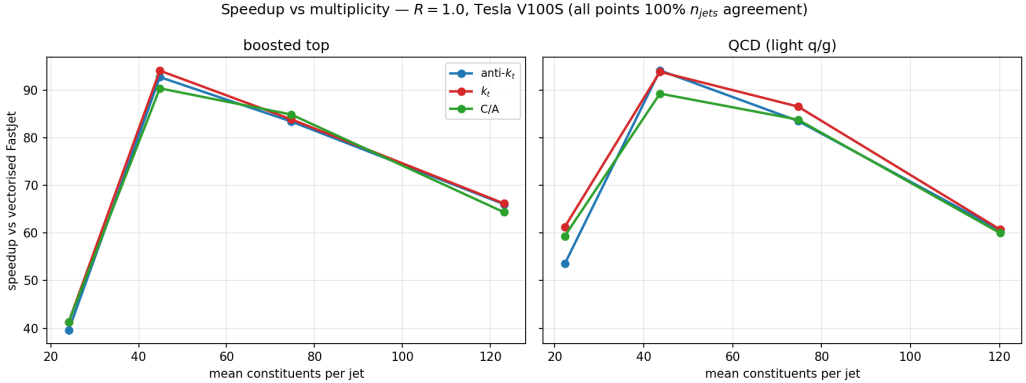
Both clusterers read the same saved file. Speedups are quoted against the **vectorised** FastJet interface — the faster, fairer baseline.

Time per jet



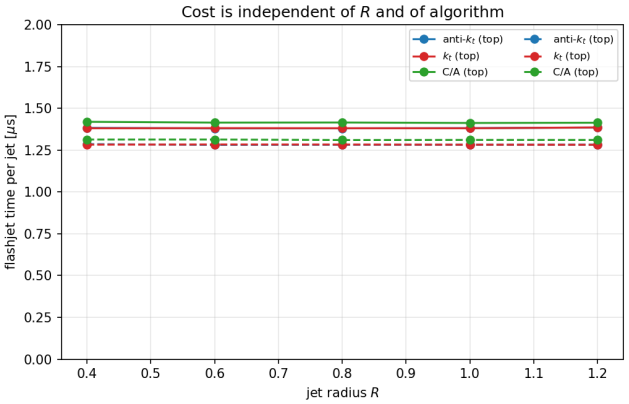
Log scale. flashjet stays nearly flat while both FastJet interfaces climb steeply with multiplicity.

The advantage grows with jet complexity



All three algorithms, both samples. Every point has **100 %** n_{jets} agreement. Busier jets → bigger win — the regime that matters for boosted-object physics.

Cost is independent of R — and of the algorithm



Across $R = 0.4 \to 1.2$: a **0.3 % spread** over a $3\times$ range in R.

R changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti-k_t, k_t and C/A agree to within $\sim$4 % at every multiplicity.

k_t and C/A timing is **free**.

The numbers

sample	alg	$\langle n \rangle$	flashjet [μs/jet]	FastJet vec. [μs/jet]	speedup
top	anti-k_t	24.1	0.427	16.5	$39\times$
top	anti-k_t	44.8	0.464	43.0	$\mathbf{93}\times$
top	C/A	74.7	1.000	84.8	$85\times$
top	C/A	123.1	2.565	165.1	$64\times$
qcd	anti-k_t	22.2	0.346	17.5	$51\times$
qcd	anti-k_t	43.6	0.463	45.7	$\mathbf{99}\times$
qcd	anti-k_t	74.7	1.215	89.8	$74\times$
qcd	anti-k_t	120.2	2.673	154.9	$58\times$

39–99$\times$ over vectorised FastJet (median $64\times$), with 100 % jet-count agreement everywhere. Benchmarked on an **NVIDIA V100-class** GPU.

$R = 1.0$ rows shown. Against the per-jet FastJet interface the gap is roughly *two orders of magnitude*.

Conclusions

Summary

1. **Clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjets, soft drop, Lund coordinates — comes free as array reads.
2. **Gives the same jets as FastJet.** 150/150 grid points at 100 % n_{jets} agreement, across anti-k_t/k_t/C-A, $R = 0.4$–1.2 and 24–123 constituents per jet. k_t and C/A validated on real data for the first time.
3. **Reproduces the experiment's own reconstructed jets** exactly, at ∼600 clusters per event.
4. **39–99× faster** than vectorised FastJet, and the advantage *grows* with jet complexity. Cost is independent of R and of algorithm.
5. Still a **prototype**, with meaningful headroom left.

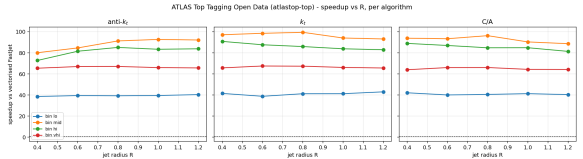
Thank you

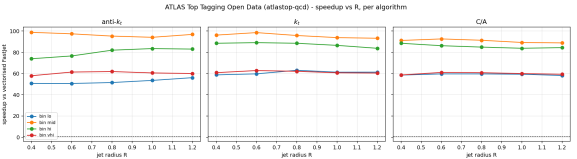
Questions?

Benchmarks: ATLAS Open Data (CC0).
Physics closures: analytic predictions on toy showers.

Backup

Backup — speedup vs R, per algorithm



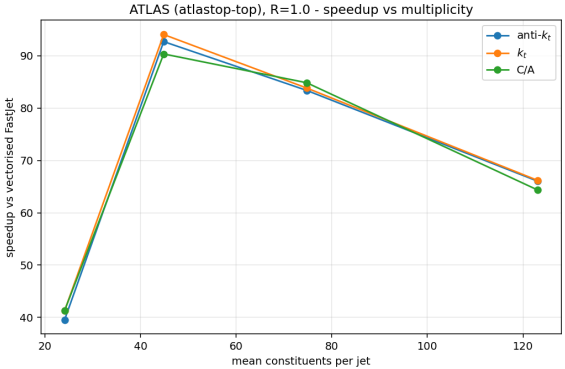


boosted top

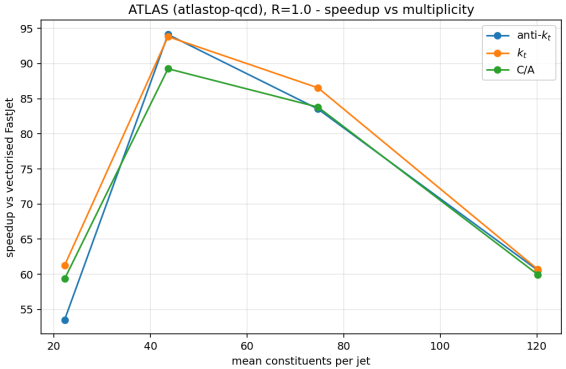
QCD (light q/g)

Flat in R for all three algorithms; ordering is by multiplicity bin.

Backup — speedup vs multiplicity



boosted top



QCD (light q/g)

Backup — the datasets

All benchmark data is **ATLAS Open Data** (CC0), read over the public redirector — no grid certificate required.

dataset	used for
ATLAS Top Tagging Open Data	timing + FastJet agreement
2020 ATLAS Jet Reconstruction	closure vs the experiment's own jets

Constituents are calorimeter clusters, massless. Physics closures use toy showers compared against leading-log analytic predictions, so the prediction being tested is unambiguous.